

OnePAY REST Webservice Signature

Interface Specifications

OnePAY Technical Team

12/4/2016

Confidential Document, only use internal OnePAY Technical Team

History of changes

[illegible]

Contents

[Authenticating Requests: Using the X-OP-Authorization Header \(OWS Signature Version 1\)](#)

[Overview](#)

[Signature Calculations for the X-OP-Authorization Header: Transferring Payload in a Single Chunk \(OWS Signature Version 1\)](#)

[Calculating a Signature](#)

[Task 1: Create a Canonical Request](#)

[Task 2: Create a String to Sign](#)

[Task 3: Calculate Signature](#)

[Examples: Signature Calculations](#)

[Example: GET Object](#)

[Example: PUT Object](#)

[Example: GET Bucket Lifecycle](#)

[Example: Get Bucket \(List Objects\)](#)

[Signature Calculations for the X-OP-Authorization Header: Transferring Payload in Multiple Chunks \(Chunked Upload\) \(OWS Signature Version 1\)](#)

[Calculating the Seed Signature](#)

[Defining the Chunk Body](#)

[Example: PUT Object](#)

[Example: PUT Object](#)

[Authenticating Requests: Using Query Parameters \(OWS Signature Version 1\)](#)

[Calculating a Signature](#)

[An Example](#)

Authenticating Requests: Using the X-OP-Authorization Header (OWS Signature Version 1)

Topics

- Overview
- Signature Calculations for the X-OP-Authorization Header: Transferring Payload in a Single Chunk (OWS Signature Version 1)
- Signature Calculations for the Authorization Header: Transferring Payload in Multiple Chunks (Chunked Upload) (OWS Signature Version 1)

Overview

Using the HTTP `X-OP-Authorization` header is the most common method of providing authentication information. Except for POST requests and requests that are signed by using query parameters, all OnePAY REST operations use the `X-OP-Authorization` request header to provide authentication information.

The following is an example of the `X-OP-Authorization` header value. Line breaks are added to this example for readability:

```
X-OP-Authorization: OWS1-HMAC-SHA256  
Credential=AKIAIOSFODNN7EXAMPLE/20160401/onepay/tsp/ows1_request,  
SignedHeaders=host;range;x-op-date,  
Signature=fe5f80f77d5fa3beca038a248ff027d0445342fe2855ddc963176630326f1024
```

The following is the properly formatted version of the same Authorization header:

Note the following:

- There is space between the first two components, `OWS1-HMAC-SHA256` and `Credential`
- The subsequent components, `Credential`, `SignedHeaders`, and `Signature` are separated by a comma.

The following table describes the various components of the `X-OP-Authorization` header value in the preceding example:

Component	Description
OWS1-HMAC-SHA256	The algorithm that was used to calculate the signature. You must provide this value when you use OWS Signature Version 1 for authentication. The string specifies OWS Signature Version 1 (<code>ows1</code>) and the signing algorithm (<code>HMAC-SHA256</code>).

Credential	Your access key ID and the scope information, which includes the date, region, and service that were used to calculate the signature. This string has the following form: <pre><your-access-key-id>/<date>/<ows-region>/<ows-service>/o wsl_request</pre> Where: • <code><date></code> value is specified using YYYYMMDD format. • <code><ows-service></code> value is <code>tsp</code> when sending request to Token Service Provider.
SignedHeaders	A semicolon-separated list of request headers that you used to compute Signature. The list includes header names only, and the header names must be in lowercase. For example: <pre>host;range;x-op-date</pre>
Signature	The 256-bit signature expressed as 64 lowercase hexadecimal characters. For example: <pre>fe5f80f77d5fa3beca038a248ff027d0445342fe2855ddc9631766303 26f1024</pre> Note that the signature calculations vary depending on the option you choose to transfer the payload.

The signature calculations vary depending on the method you choose to transfer the request payload.

TSP supports the following options:

- **Transfer payload in a single chunk** – In this case, you have the following signature calculation options:
 - **Signed payload option** – You can optionally compute the entire payload checksum and include it in signature calculation. This provides added security but you need to read your payload twice or buffer it in memory.
 - For example, in order to upload a file, you need to read the file first to compute a payload hash for signature calculation and again for transmission when you create the request. For smaller payloads, this approach might be preferable. However, for large files, reading the file twice can be inefficient, so you might want to upload data in chunks instead.
 - We recommend you include payload checksum for added security.
 - **Unsigned payload option** – Do not include payload checksum in signature calculation.

- For step-by-step instructions to calculate signature and construct the X-OP-Authorization header value, see [Signature Calculations for the Authorization Header: Transferring Payload in a Single Chunk \(OWS Signature Version 1\)](#).
- **Transfer payload in multiple chunks (chunked upload)** – In this case you transfer payload in chunks. You can transfer a payload in chunks regardless of the payload size.
- You can break up your payload into chunks. These can be fixed or variable-size chunks. By uploading data in chunks, you avoid reading the entire payload to calculate the signature. Instead, for the first chunk, you calculate a seed signature that uses only the request headers. The second chunk contains the signature for the first chunk, and each subsequent chunk contains the signature for the chunk that precedes it. At the end of the upload, you send a final chunk with 0 bytes of data that contains the signature of the last chunk of the payload. For more information, see [Signature Calculations for the X-OP-Authorization Header: Transferring Payload in Multiple Chunks \(Chunked Upload\) \(OWS Signature Version 1\)](#).

When you send a request, you must tell TSP which of the preceding options you have chosen in your signature calculation, by adding the `x-op-content-sha256` header with one of the following values:

- If you choose chunked upload options, set the header value to `STREAMING-OWS1-HMAC-SHA256-PAYLOAD`.
- If you choose to upload payload in a single chunk, set the header value to the payload checksum (signed payload option), or set the value to the literal string `UNSIGNED-PAYLOAD` (unsigned payload option).

Upon receiving the request, TSP re-creates the string to sign using information in the `X-OP-Authorization` header and the `date` header. It then verifies with authentication service the signatures match. The request date can be specified by using either the HTTP `Date` or the `x-op-date` header. If both headers are present, `x-op-date` takes precedence.

If the signatures match, TSP processes your request; otherwise, your request will fail.

For more information, see the following topics:

[Signature Calculations for the Authorization Header: Transferring Payload in a Single Chunk \(OWS Signature Version 1\)](#)

[Signature Calculations for the Authorization Header: Transferring Payload in Multiple Chunks \(Chunked Upload\) \(OWS Signature Version 1\)](#)

Signature Calculations for the X-OP-Authorization Header: Transferring Payload in a Single Chunk (OWS Signature Version 1)

When using the `X-OP-Authorization` header to authenticate requests, the header value includes, among other things, a signature. The signature calculations vary depending on the choice you make for transferring the payload (Overview). This section explains signature calculations when you choose to transfer the payload in a single chunk. The example section (see Examples: Signature Calculations) shows signature calculations and resulting `X-OP-Authorization` headers that you can use as a test suite to verify your code.

Important

When transferring payload in a single chunk, you can optionally choose to include the payload hash in the signature calculations, referred as *signed payload* (if you don't include it, the payload is considered *unsigned*). The signing procedure discussed in the following section applies to both, but note the following differences:

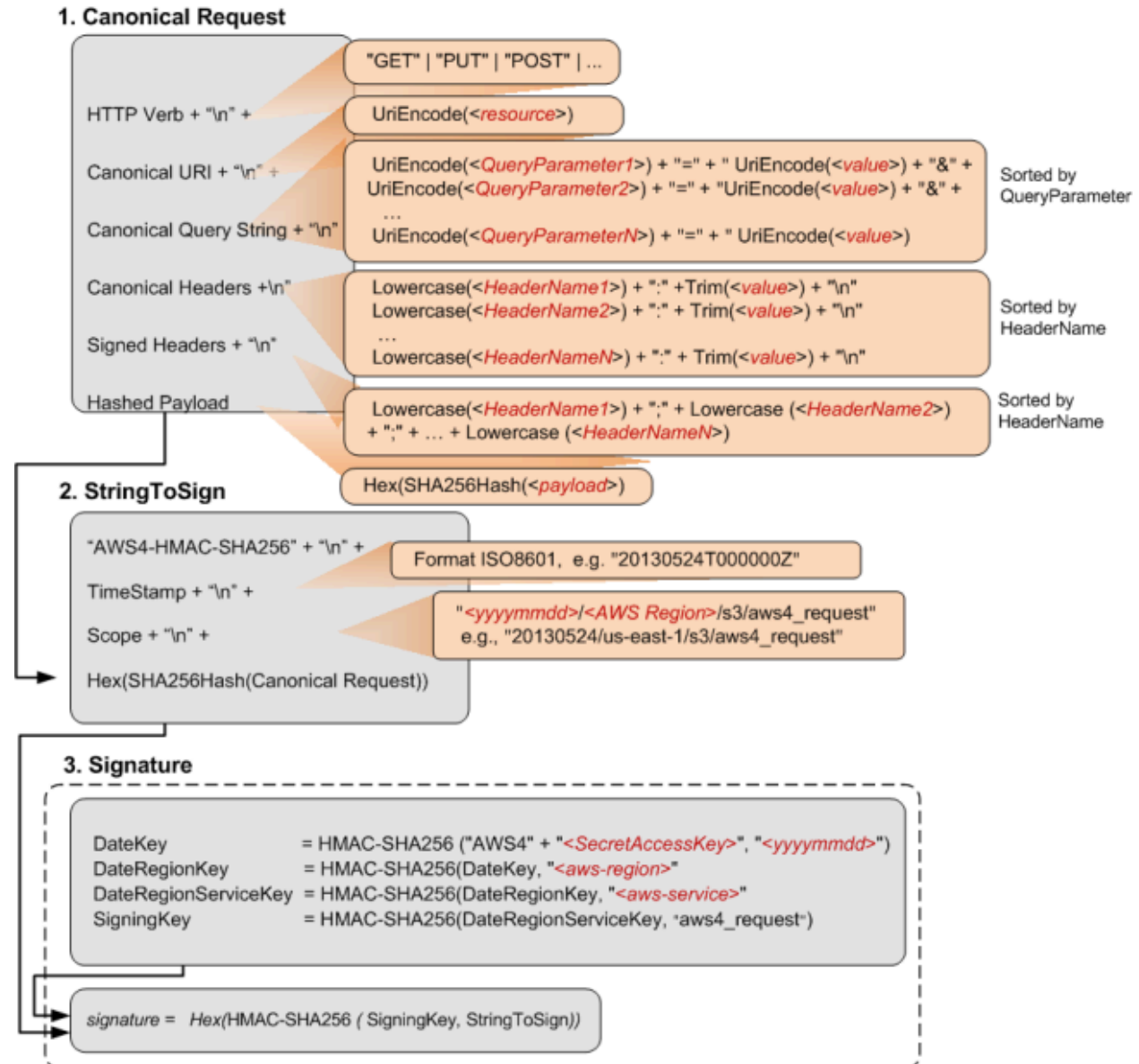
- **Signed payload option** – You include the payload hash when constructing the canonical request (that then becomes part of `StringToSign`, as explained in the signature calculation section). You also specify the same value as the `x-op-content-sha256` header value when sending the request to TSP.
- **Unsigned payload option** – You include the literal string `UNSIGNED-PAYLOAD` when constructing a canonical request, and set the same value as the `x-op-content-sha256` header value when sending the request to TSP.

When you send your request to TSP, the `x-op-content-sha256` header value informs TSP whether the payload is signed or not. TSP can then create signature accordingly for verification.

Calculating a Signature

To calculate a signature, you first need a string to sign. You then calculate a `HMAC-SHA256` hash of the string to sign by using a signing key. The following diagram illustrates the process, including the various components of the string that you create for signing

When TSP receives an authenticated request, it computes the signature and then compares it with the signature that you provided in the request. For that reason, you must compute the signature by using the same method that is used by TSP. The process of putting a request in an agreed-upon form for signing is called canonicalization.



The following table describes the functions that are shown in the diagram. You need to implement code for these functions.

Function	Description
Lowercase ()	Convert the string to lowercase.

Hex ()	Lowercase base 16 encoding.
SHA256Hash ()	Secure Hash Algorithm (SHA) cryptographic hash function.
HMAC-SHA256 ()	Computes HMAC by using the SHA256 algorithm with the signing key provided. This is the final signature.
Trim ()	Remove any leading or trailing whitespace.
UriEncode ()	URI encode every byte. UriEncode() must enforce the following rules: • URI encode every byte except the unreserved characters: 'A'-'Z', 'a'-'z', '0'-'9', '-', '.', '_', and '~'. • The space character is a reserved character and must be encoded as "%20" (and not as "+"). • Each URI encoded byte is formed by a '%' and the two-digit hexadecimal value of the byte. • Letters in the hexadecimal value must be uppercase, for example "%1A". • Encode the forward slash character, '/', everywhere except in the object key name. For example, if the object key name is <code>photos/Jan/sample.jpg</code>, the forward slash in the key name is not encoded. Caution The standard UriEncode functions provided by your development platform may not work because of differences in implementation and related ambiguity in the underlying RFCs. We recommend that you write your own custom UriEncode function to ensure that your encoding will work. The following is an example uri-encode() function in Java. <pre> public static String UriEncode(CharSequence input, boolean encodeSlash) { StringBuilder result = new StringBuilder(); for (int i = 0; i < input.length(); i++) { char ch = input.charAt(i); if ((ch >= 'A' && ch <= 'Z') (ch >= 'a' && ch <= 'z') (ch >= '0' && ch <= '9') ch == '_' ch == '-' ch == '~' ch == '.') { </pre>

	<pre> result.append(ch); } else if (ch == '/') { result.append(encodeURIComponent(ch)); } else { result.append(toHexUTF8(ch)); } } return result.toString(); } </pre>
--	---

Task 1: Create a Canonical Request

This section provides an overview of creating a canonical request.

The following is the canonical request format that TSP uses to calculate a signature. For signatures to match, you must create a canonical request in this format:

```

<HTTPMethod>\n
<CanonicalURI>\n
<CanonicalQueryString>\n
<CanonicalHeaders>\n
<SignedHeaders>\n
<HashedPayload>

```

Where:

- *HTTPMethod* is one of the HTTP methods, for example GET, PUT, HEAD, and DELETE.
- *CanonicalURI* is the URI-encoded version of the absolute path component of the URI—everything starting with the "/" that follows the domain name and up to the end of the string or to the question mark character (?) if you have query string parameters. The URI in the following example, /example/resources, is the absolute path and you don't encode the "/" in the absolute path:
 - `https://onipay.vn/example/resources`
- *CanonicalQueryString* specifies the URI-encoded query string parameters. You URI-encode name and values individually. You must also sort the parameters in the canonical query string alphabetically by key name. The sorting occurs after encoding. The query string in the following URI example is `prefix=somePrefix&marker=someMarker&max-keys=20`:
 - `http://onipay.vn/example?prefix=somePrefix&marker=someMarker&max-keys=20`
- The canonical query string is as follows (line breaks are added to this example for readability):
 - `URI-encode("marker")+"="+URI-encode("someMarker")+"&"+`
`URI-encode("max-keys")+"="+URI-encode("20") + "& " +`
`URI-encode("prefix")+"="+URI-encode("somePrefix")`
- When a request targets a subresource, the corresponding query parameter value will be an empty string (""). For example, the following URI identifies the ACL subresource on the example:
 - `http://onipay.vn/examplebucket?acl`

- The CanonicalQueryString in this case is as follows:
- `URI-encode("acl") + "=" + ""`
- If the URI does not include a '?', there is no query string in the request, and you set the canonical query string to an empty string (""). You will still need to include the "\n".
- *CanonicalHeaders* is a list of request headers with their values. Individual header name and value pairs are separated by the newline character ("\n"). Header names must be in lowercase. You must sort the header names alphabetically to construct the string, as shown in the following example:
- `Lowercase(<HeaderName1>) + ":" + Trim(<value>) + "\n"`
`Lowercase(<HeaderName2>) + ":" + Trim(<value>) + "\n"`
`...`
`Lowercase(<HeaderNameN>) + ":" + Trim(<value>) + "\n"`
- The `Lowercase()` and `Trim()` functions used in this example are described in the preceding section.
- The *CanonicalHeaders* list must include the following:
 - HTTP host header.
 - If the Content-Type header is present in the request, you must add it to the *CanonicalHeaders* list.
 - Any x-op-* headers that you plan to include in your request must also be added. For example, if you are using temporary security credentials, you need to include x-op-security-token in your request. You must add this header in the list of *CanonicalHeaders*.
- **Note**
- The x-op-content-sha256 header is required for all OWS Signature Version 1 requests. It provides a hash of the request payload. If there is no payload, you must provide the hash of an empty string.
- The following is an example CanonicalHeaders string. The header names are in lowercase and sorted.
- `host:onepay.vn`
`x-op-content-sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855`
`x-op-date:20130708T220855Z`
- **Note**
- For the purpose of calculating an authorization signature, only the host and any x-op-* headers are required; however, in order to prevent data tampering, you should consider including all the headers in the signature calculation.

- *SignedHeaders* is an alphabetically sorted, semicolon-separated list of lowercase request header names. The request headers in the list are the same headers that you included in the CanonicalHeaders string. For example, for the previous example, the value of *SignedHeaders* would be as follows:
- `host;x-op-content-sha256;x-op-date`
- *HashedPayload* is the hexadecimal value of the SHA256 hash of the request payload.
- `Hex (SHA256Hash (<payload>))`
- If there is no payload in the request, you compute a hash of the empty string as follows:
- `Hex (SHA256Hash (""))`
- The hash returns the following value:
- `e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855`
- For example, when you upload an object by using a PUT request, you provide object data in the body. When you retrieve an object by using a GET request, you compute the empty string hash.

Task 2: Create a String to Sign

This section provides an overview of creating a string to sign. For step-by-step instructions, see Task 2: Create a String to Sign in the *OWS General Reference*.

The string to sign is a concatenation of the following strings:

```
"OWS1-HMAC-SHA256" + "\n" +
timeStampISO8601Format + "\n" +
<Scope> + "\n" +
Hex (SHA256Hash (<CanonicalRequest>))
```

The constant string OWS1-HMAC-SHA256 specifies the hash algorithm that you are using, HMAC-SHA256. The `timeStamp` is the current UTC time in ISO 8601 format (for example, 20160401T000000Z).

`Scope` binds the resulting signature to a specific date, an OWS region, and a service. Thus, your resulting signature will work only in the specific region and for a specific service. The signature is valid for seven days after the specified date.

```
date.Format (<YYYYMMDD>) + "/" + <region> + "/" + <service> + "/ows1_request"
```

For TSP, the service string is `tsp`. For a list of *region* strings, see Regions and Endpoints in the *OWS General Reference*. The region column in this table provides the list of valid region strings.

The following scope restricts the resulting signature to the `onipay` region and TSP.

```
20130606/onipay/tsp/ows1_request
```

Note

`Scope` must use the same date that you use to compute the signing key, as discussed in the following section.

Task 3: Calculate Signature

In OWS Signature Version 1, instead of using your OWS access keys to sign a request, you first create a signing key that is scoped to a specific region and service. For more information about signing keys, see [Introduction to Signing Requests](#).

```
DateKey          = HMAC-SHA256 ("OWS1"+"<SecretAccessKey>", "<YYYYMMDD>")
DateRegionKey    = HMAC-SHA256 (<DateKey>, "<aws-region>")
DateRegionServiceKey = HMAC-SHA256 (<DateRegionKey>, "<aws-service>")
SigningKey       = HMAC-SHA256 (<DateRegionServiceKey>, "ows1_request")
```

Note

This signing key is valid for seven days from the date specified in the `DateKey` hash.

For a list of region strings, see Regions and Endpoints in the *OWS General Reference*.

Using a signing key enables you to keep your OWS credentials in one safe place. For example, if you have multiple servers that communicate with TSP, you share the signing key with those servers; you don't have to keep a copy of your secret access key on each server. Signing key is valid for up to seven days. So each time you calculate signing key you will need to share the signing key with your servers. For more information, see [Authenticating Requests \(OWS Signature Version 1\)](#).

The final signature is the HMAC-SHA256 hash of the string to sign, using the signing key as the key.

```
HMAC-SHA256(SigningKey, StringToSign)
```

For step-by-step instructions on creating a signature, see [Task 3: Create a Signature](#) in the *OWS General Reference*.

Examples: Signature Calculations

You can use the examples in this section as a reference to check signature calculations in your code. For additional references, see Signature Version 1 Test Suite of the *OWS General Reference*. The calculations shown in the examples use the following data:

- Example access keys.

Parameter	Value
OWSAccessKeyId	AKIAIOSFODNN7EXAMPLE
OWSSecretAccessKey	wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY

- Request timestamp of 20160401T000000Z (Fri, 01 Apr 2016 00:00:00 GMT).
- Bucket name `examplebucket`.

- The bucket is assumed to be in the US East (N. Virginia) region. The credential `Scope` and the `Signing Key` calculations use `oneway` as the region specifier. For information about other regions, see *Regions and Endpoints in the OWS General Reference*.
- You can use either path-style or virtual hosted-style requests. The following examples show how to sign a virtual hosted-style request, for example:
 - `https://secure.oneway.vn/photos/photo1.jpg`
 - For more information, see *Virtual Hosting of Buckets in the Amazon Simple Storage Service Developer Guide*.

Example: GET Object

The following example gets the first 10 bytes of an object (`test.txt`) from `examplebucket`. For more information about the API action, see *GET Object*.

```
GET /test.txt HTTP/1.1
Host: secure.oneway.vn
x-op-date:20160401T000000Z
Authorization: SignatureToBeCalculated
Range: bytes=0-9
x-op-content-sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
x-op-date: 20160401T000000Z
```

Because this GET request does not provide any body content, the `x-op-content-sha256` value is the hash of the empty request body. The following steps show signature calculations and construction of the `X-OP-Authorization` header.

1. StringToSign

1. CanonicalRequest

2. GET

```
/test.txt
```

```
host:secure.oneway.vn
```

```
range:bytes=0-9
```

```
x-op-content-sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
```

```
x-op-date:20160401T000000Z
```

```
host;range;x-op-content-sha256;x-op-date
```

```
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
```

- ##### 3. In the canonical request string, the last line is the hash of the empty request body. The third line is empty because there are no query parameters in the request.

4. StringToSign

5. OWS1-HMAC-SHA256

```
20160401T000000Z
```

```
20160401/onepay/tsp/ows1_request
7344ae5b7ee6c3e7e6b0fe0640412a37625d1fbfff95c48bbb2dc43964946972
```

2. SigningKey

3. `signing key = HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 ("OWS1" + "<YourSecretAccessKey>", "20160401"), "onepay"), "tsp"), "ows1_request")`

4. Signature

5. `f0e8bdb87c964420e857bd35b5d6ed310bd44f0170aba48dd91039c6036bdb41`

6. Authorization header

7. The resulting Authorization header is as follows:

8. `OWS1-HMAC-SHA256
Credential=AKIAIOSFODNN7EXAMPLE/20160401/onepay/tsp/ows1_request,SignedHe
aders=host;range;x-op-content-sha256;x-op-date,Signature=f0e8bdb87c964420
e857bd35b5d6ed310bd44f0170aba48dd91039c6036bdb41`

Example: PUT Object

This example PUT request creates an object (`test$file.text`) in `examplebucket`. The example assumes the following:

- You are requesting `REDUCED_REDUNDANCY` as the storage class by adding the `x-op-storage-class` request header. For information about storage classes, see *Storage Classes in the Amazon Simple Storage Service Developer Guide*.
- The content of the uploaded file is a string, "Welcome to TSP." The value of `x-op-content-sha256` in the request is based on this string.

For information about the API action, see `PUT Object`.

```
PUT test$file.text HTTP/1.1
Host: secure.onepay.vn
Date: Fri, 01 Apr 2016 00:00:00 GMT

X-OP-Authorization: SignatureToBeCalculated
x-op-date: 20160401T000000Z
x-op-storage-class: REDUCED_REDUNDANCY
x-op-content-sha256:
44ce7dd67c959e0d3524ffac1771dfbba87d2b6b4b4e99e42034a8b803f8b072
```

<Payload>

The following steps show signature calculations.

1. StringToSign

1. CanonicalRequest

2. PUT

```
/test%24file.text
```

```
date:Fri, 01 Apr 2016 00:00:00 GMT
host:secure.onepay.vn
```

```
x-op-content-sha256:44ce7dd67c959e0d3524ffac1771dfbba87d2b6b4b4e99e42034a8b803f8b072
x-op-date:20160401T000000Z
x-op-storage-class:REDUCED_REDUNDANCY
```

```
date;host;x-op-content-sha256;x-op-date;x-op-storage-class
44ce7dd67c959e0d3524ffac1771dfbba87d2b6b4b4e99e42034a8b803f8b072
```

3. In the canonical request, the third line is empty because there are no query parameters in the request. The last line is the hash of the body, which should be same as the x-op-content-sha256 header value.

4. **StringToSign**

5. OWS1-HMAC-SHA256
20160401T000000Z
20160401/onepay/tsp/ows1_request
9e0e90d9c76de8fa5b200d8c849cd5b8dc7a3be3951ddb7f6a76b4158342019d

2. **SigningKey**

3. `signing key = HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 ("OWS1" + "<YourSecretAccessKey>", "20160401"), "onepay"), "tsp"), "ows1_request")`

4. **Signature**

5. 98ad721746da40c64f1a55b78f14c238d841ea1380cd77a1b5971af0ece108bd

6. **Authorization header**

7. The resulting Authorization header is as follows:

8. OWS1-HMAC-SHA256
Credential=AKIAIOSFODNN7EXAMPLE/20160401/onepay/tsp/ows1_request,SignedHeaders=date;host;x-op-content-sha256;x-op-date;x-op-storage-class,Signature=98ad721746da40c64f1a55b78f14c238d841ea1380cd77a1b5971af0ece108bd

Example: GET Bucket Lifecycle

The following GET request retrieves the lifecycle configuration of `examplebucket`. For information about the API action, see GET Bucket lifecycle.

```
GET ?lifecycle HTTP/1.1
Host: secure.onepay.vn
Authorization: SignatureToBeCalculated
x-op-date: 20160401T000000Z
x-op-content-sha256:e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
```

Because the request does not provide any body content, the x-op-content-sha256 header value is the hash of the empty request body. The following steps show signature calculations.

1. **StringToSign**

1. **CanonicalRequest**

2. GET
/
lifecycle=


```
host:secure.onepay.vn
x-op-content-sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934
ca495991b7852b855
x-op-date:20160401T000000Z
```

```
host;x-op-content-sha256;x-op-date
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
```

3. In the canonical request, the last line is the hash of the empty request body.

4. **StringToSign**

5. OWS1-HMAC-SHA256
20160401T000000Z
20160401/onepay/tsp/ows1_request
9766c798316ff2757b517bc739a67f6213b4ab36dd5da2f94eaebf79c77395ca

2. **SigningKey**

3. *signing key* = HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 ("OWS1" + "<YourSecretAccessKey>", "20160401"), "onepay"), "tsp"), "ows1_request")

4. **Signature**

5. fea454ca298b7da1c68078a5d1bdbfbbe0d65c699e0f91ac7a200a0136783543

6. **Authorization header**

7. The resulting Authorization header is as follows:

8. OWS1-HMAC-SHA256
Credential=AKIAIOSFODNN7EXAMPLE/20160401/onepay/tsp/ows1_request, SignedHe
aders=host;x-op-content-sha256;x-op-date, Signature=fea454ca298b7da1c68078
a5d1bdbfbbe0d65c699e0f91ac7a200a0136783543

Example: Get Bucket (List Objects)

The following example retrieves a list of objects from `examplebucket` bucket. For information about the API action, see [GET Bucket \(List Objects\)](#).

```
GET ?max-keys=2&prefix=J HTTP/1.1
Host: secure.onepay.vn
Authorization: SignatureToBeCalculated
x-op-date: 20160401T000000Z
x-op-content-sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b785
2b855
```

Because the request does not provide a body, the value of `x-op-content-sha256` is the hash of the empty request body. The following steps show signature calculations.

1. **StringToSign**

1. **CanonicalRequest**

2. GET

```
/
max-keys=2&prefix=J
host:onepay.vn
x-op-content-sha256:e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934
```

```
ca495991b7852b855
x-op-date:20160401T000000Z
```

```
host;x-op-content-sha256;x-op-date
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
```

3. In the canonical string, the last line is the hash of the empty request body.

4. **StringToSign**

5. OWS1-HMAC-SHA256
20160401T000000Z
20160401/onepay/tsp/ows1_request
df57d21db20da04d7fa30298dd4488ba3a2b47ca3a489c74750e0f1e7df1b9b7

2. **SigningKey**

3. `signing key = HMAC-SHA256(HMAC-SHA256(HMAC-SHA256(HMAC-SHA256("OWS1" + "<YourSecretAccessKey>", "20160401"), "onepay"), "tsp"), "ows1_request")`

4. **Signature**

5. 34b48302e7b5fa45bde8084f4b7868a86f0a534bc59db6670ed5711ef69dc6f7

6. **Authorization header**

7. The resulting Authorization header is as follows:

8. OWS1-HMAC-SHA256
Credential=AKIAIOSFODNN7EXAMPLE/20160401/onepay/tsp/ows1_request,SignedHe
aders=hos

Signature Calculations for the X-OP-Authorization Header: Transferring Payload in Multiple Chunks (Chunked Upload) (OWS Signature Version 1)

As described in the Overview, when authenticating requests using the `Authorization` header, you have an option of uploading the payload in chunks. You can send data in fixed size or variable size chunks. This section describes the signature calculation process in chunked upload, how you create the chunk body, and how the delayed signing works where you first upload the chunk, and send its signature in the subsequent chunk. The example section (see Example: PUT Object) shows signature calculations and resulting `Authorization` headers that you can use as test suite to verify your code.

Note

When transferring data in a series of chunks, you must use the `Content-Length` HTTP header to explicitly specify the total content length (object length in bytes plus metadata in each chunk). This will require you to pre-compute the total length of the payload including the metadata you will send in each chunk before starting your request. The `x-op-decoded-content-length` header will contain the size of the object length in bytes.

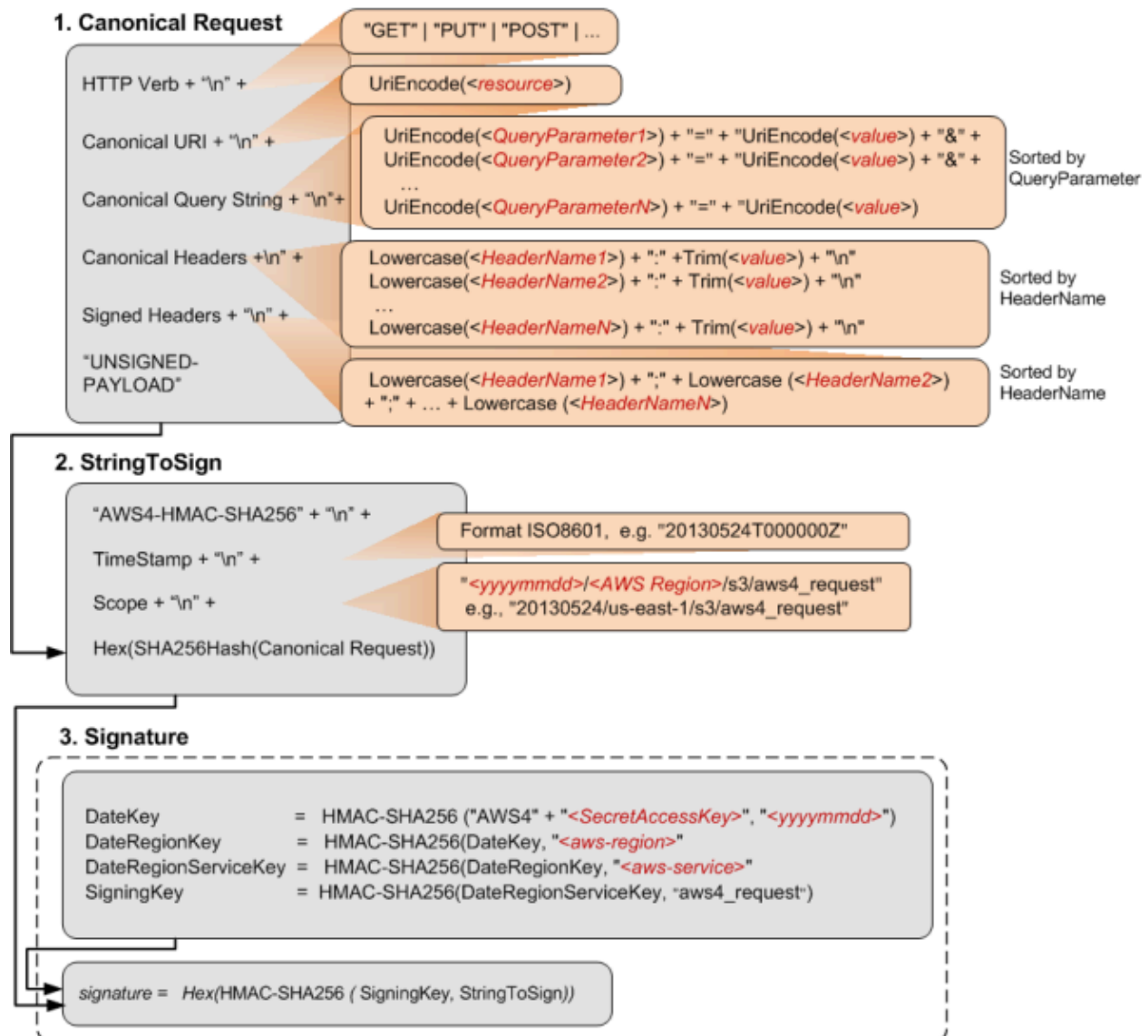
Each chunk signature calculation includes the signature of the previous chunk. To begin with, you create a *seed* signature using only the headers. You use the seed signature in the signature calculation of the first chunk. For each subsequent chunk, you create a chunk signature that includes signature of the previous chunk. Thus, the chunk signatures are chained together; that is, signature of chunk n is a function $F(\text{chunk } n, \text{signature}(\text{chunk } n-1))$. The chaining ensures you send the chunks in correct order. To perform a chunked upload, do the following:

1. Decide payload chunk size. You need this when you write the code.
2. Chunk size must be at least 8 KB. We recommend a chunk size of at least 64 KB for better performance. This chunk size applies to all chunk except the last one. The last chunk you send can be smaller than 8 KB. If your payload is small and can fit in one chunk, then it can be smaller than the 8 KB.
3. Create the seed signature for inclusion in the first chunk. For more information, see Calculating the Seed Signature.
4. Create the first chunk and stream it. For more information, see Defining the Chunk Body.

- For each subsequent chunk, calculate the chunk signature that includes the previous signature in the string you sign, construct the chunk and send it. For more information, see Defining the Chunk Body.
- Send the final additional chunk, same as other chunks in construction, but it has zero data bytes. For more information, see Defining the Chunk Body.

Calculating the Seed Signature

The following diagram illustrates the process of calculating the seed signature.



The following table describes the functions that are shown in the diagram. You need to implement code for these functions.

Function	Description
<code>Lowercase()</code>	Convert the string to lowercase.
<code>Hex()</code>	Lowercase base 16 encoding.
<code>SHA256Hash()</code>	Secure Hash Algorithm (SHA) cryptographic hash function.
<code>HMAC-SHA256()</code>	Computes HMAC by using the SHA256 algorithm with the signing key provided. This is the final signature.
<code>Trim()</code>	Remove any leading or trailing whitespace.
<code>UriEncode()</code>	URI encode every byte. <code>UriEncode()</code> must enforce the following rules: • URI encode every byte except the unreserved characters: 'A'-'Z', 'a'-'z', '0'-'9', '-', '.', '_', and '~'. • The space character is a reserved character and must be encoded as "%20" (and not as "+"). • Each URI encoded byte is formed by a '%' and the two-digit hexadecimal value of the byte. • Letters in the hexadecimal value must be uppercase, for example "%1A". • Encode the forward slash character, '/', everywhere except in the object key name. For example, if the object key name is <code>photos/Jan/sample.jpg</code>, the forward slash in the key name is not encoded. Caution The standard <code>UriEncode</code> functions provided by your development platform may not work because of differences in implementation and related ambiguity in the underlying RFCs. We recommend that you write your own custom <code>UriEncode</code> function to ensure that your encoding will work. The following is an example <code>uri-encode()</code> function in Java.

```
public static String UriEncode(CharSequence input, boolean
encodeSlash) {
    StringBuilder result = new StringBuilder();
    for (int i = 0; i < input.length(); i++) {
        char ch = input.charAt(i);
        if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' &&
ch <= 'z') || (ch >= '0' && ch <= '9') || ch == '_' || ch ==
'-' || ch == '~' || ch == '.') {
            result.append(ch);
        } else if (ch == '/') {
            result.append(encodeSlash ? "%2F" : ch);
        } else {
            result.append(toHexUTF8(ch));
        }
    }
    return result.toString();
}
```

For information about the signing process, see [Signature Calculations for the Authorization Header: Transferring Payload in a Single Chunk \(OWS Signature Version 1\)](#). The process is the same except that the creation of `CanonicalRequest` differs as follows:

- In addition to the request headers you plan to add, you must include the following headers:

Header	Description
x-op-content-sha256	This header is required for all OWS Signature Version 1 requests. Set the value <code>toSTREAMING-OWS1-HMAC-SHA256-PAYLOAD</code> to indicate that the signature covers only headers and that there is no payload.
Content-Encoding	Set the value to <code>aws-chunked</code>. TSP supports multiple content encodings. For example: <pre>Content-Encoding : aws-chunked,gzip</pre> That is, you can specify your custom content-encoding when using Signature Version 1 streaming API. Note

	TSP will store the resulting object without the <code>aws-chunked</code> encoding. Therefore, when you retrieve the object it will not be <code>aws-chunked</code> encoded.
<code>x-op-decoded-content-length</code>	Set the value to the length, in bytes, of the data to be chunked, without counting any metadata. For example, if you are uploading a 4 GB file, set the value to 4294967296.
<code>Content-Length</code>	Set the value to the length of your data, including the metadata. Each chunk will have metadata, such as the signature of the previous chunk. Chunk calculations are discussed in the following section.

You send the first chunk with the seed signature. You will need to construct the chunk as described in the following section.

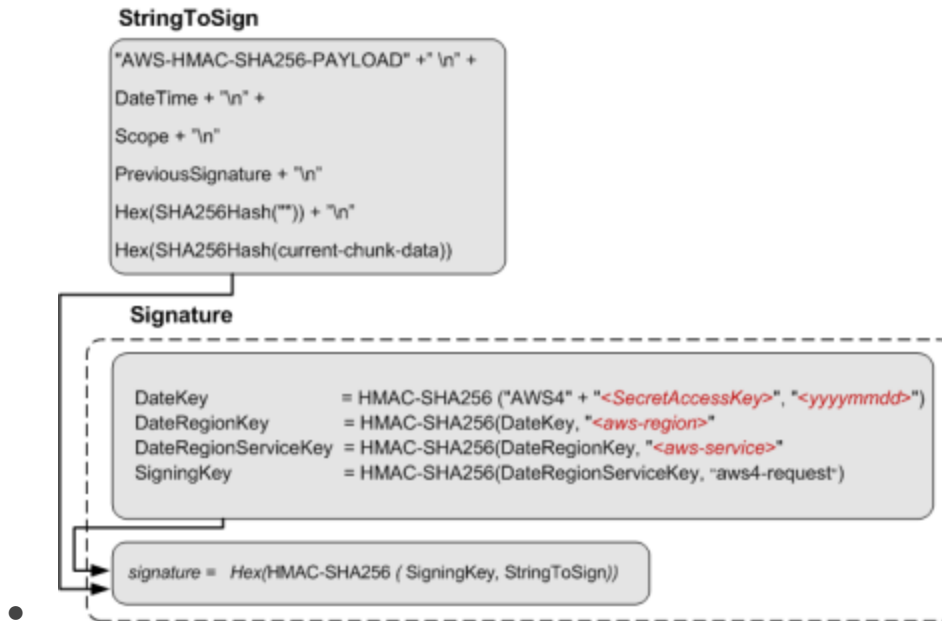
Defining the Chunk Body

All chunks include some metadata. Each chunk must conform to the following structure:

```
string(IntHexBase(chunk-size)) + ";chunk-signature=" + signature + \r\n +
chunk-data + \r\n
```

Where:

- `IntHexBase()` is a function that you will write to convert an integer `chunk-size` to hexadecimal. For example, if `chunk-size` is 65536, hexadecimal string is "1000".
- *chunk-size* is the size, in bytes, of the `chunk-data`, without metadata. For example, if you are uploading a 65 KB object and using a chunk size of 64 KB, you upload the data in three chunks: the first would be 64 KB, the second 1 KB, and the final chunk with 0 bytes.
- *signature* For each chunk, you calculate signature using the following string to sign. For the first chunk, you use the seed-signature as the previous signature.



The size of the final chunk data that you send is 0, although the chunk body will still contain metadata, including the signature of the previous chunk.

Example: PUT Object

You can use the examples in this section as a reference to check signature calculations in your code. Before you review the examples, note the following:

- The signature calculations in these examples use the following example security credentials.

Parameter	Value
OWSAccessKeyId	AKIAIOSFODNN7EXAMPLE
OWSSecretAccessKey	wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY

- All examples use the request timestamp 20160401T000000Z (Fri, 01 Apr 2016 00:00:00 GMT).
- All examples use `examplebucket` as the bucket name.
- The bucket is assumed to be in the US East (N. Virginia) region, and the credential `Scope` and the `Signing Key` calculations use `onipay` as the region specifier. For more information, see *Regions and Endpoints in the Amazon Web Services General Reference*.
- You can use either path style or virtual-hosted style requests. The examples below show use virtual-hosted style requests, for example:
- `https://secure.onipay.vn/photos/photo1.jpg`

- For more information, see Virtual Hosting of Buckets in the *Amazon Simple Storage Service Developer Guide*.

Example: PUT Object

The following example sends a PUT request to upload an object. The signature calculations assume the following:

- You are uploading a 65 KB text file, and the file content is a one-character string made up of the letter 'a'.
- The chunk size is 64 KB. As a result, the payload will be uploaded in three chunks, 64 KB, 1 KB, and the final chunk with 0 bytes of chunk data.
- The resulting object has the key name `chunkObject.txt`.
- You are requesting `REDUCED_REDUNDANCY` as the storage class by adding the `x-op-storage-class` request header.

For information about the API action, see PUT Object. The general request syntax is as follows:

```
PUT /examplebucket/chunkObject.txt HTTP/1.1
Host: onepay.vn
x-op-date: 20160401T000000Z
x-op-storage-class: REDUCED_REDUNDANCY
Authorization: SignatureToBeCalculated
x-op-content-sha256: STREAMING-OWS1-HMAC-SHA256-PAYLOAD
Content-Encoding: aws-chunked
x-op-decoded-content-length: 66560
Content-Length: 66824
<Payload>
```

The following steps show signature calculations.

1. Seed signature — Create String to Sign

1. CanonicalRequest

2. PUT

```
/examplebucket/chunkObject.txt
```

```
content-encoding:aws-chunked
```

```
content-length:66824
```

```
host:onepay.vn
```

```
x-op-content-sha256:STREAMING-OWS1-HMAC-SHA256-PAYLOAD
```

```
x-op-date:20160401T000000Z
```

```
x-op-decoded-content-length:66560
```

```
x-op-storage-class:REDUCED_REDUNDANCY
```

```
content-encoding;content-length;host;x-op-content-sha256;x-op-date;
```

```
x-op-decoded-content-length;x-op-storage-class
```

```
STREAMING-OWS1-HMAC-SHA256-PAYLOAD
```

3. In the canonical request, the third line is empty because there are no query parameters in the request. The last line is the constant string provided as the value of the hashed Payload which should be same as the value of `x-op-content-sha256` header.

4. StringToSign

5. `OWS1-HMAC-SHA256`
`20160401T000000Z`
`20160401/onepay/tsp/ows1_request`
`cee3fed04b70f867d036f722359b0b1f2f0e5dc0efadbc082b76c4c60e316455`

6. Note

7. For information about each of line in the string to sign, see the diagram that explains seed signature calculation.

2. SigningKey

3. `signing key = HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 ("OWS1" + "<YourSecretAccessKey>", "20160401"), "onepay"), "tsp"), "ows1_request")`

4. Seed Signature

5. `4f232c4386841ef735655705268965c44a0e4690baa4adea153f7db9fa80a0a9`

6. Authorization header

7. The resulting Authorization header is as follows:

8. `OWS1-HMAC-SHA256`
`Credential=AKIAIOSFODNN7EXAMPLE/20160401/onepay/tsp/ows1_request,SignedHe`
`aders=content-encoding;content-length;host;x-op-content-sha256;x-op-date;`
`x-op-decoded-content-length;x-op-storage-class,Signature=4f232c4386841ef7`
`35655705268965c44a0e4690baa4adea153f7db9fa80a0a9`

9. Chunk 1: (65536 bytes, with value 97 for letter 'a')

1. Chunk string to sign:

2. `OWS1-HMAC-SHA256-PAYLOAD`
`20160401T000000Z`
`20160401/onepay/tsp/ows1_request`
`4f232c4386841ef735655705268965c44a0e4690baa4adea153f7db9fa80a0a9`
`e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855`
`bf718b6f653bebc184e1479f1935b8da974d701b893afcf49e701f3e2f9f9c5a`

3. Note

4. To information about each line in the string to sign, see the preceding diagram that show various components of the string to sign (for example, the last three lines are, `previous-signature`, `hash ("")`, and `hash (current-chunk-data)`).

5. Chunk signature:

6. `ad80c730a21e5b8d04586a2213dd63b9a0e99e0e2307b0ade35a65485a288648`

7. Chunk data sent:

8. `10000;chunk-signature=ad80c730a21e5b8d04586a2213dd63b9a0e99e0e2307b0ade35a65485a288648<65536-bytes>`
10. **Chunk 2: (1024 bytes, with value 97 for letter 'a')**
 1. Chunk string to sign:
 2. `OWS1-HMAC-SHA256-PAYLOAD
20160401T000000Z
20160401/onepay/tsp/ows1_request
ad80c730a21e5b8d04586a2213dd63b9a0e99e0e2307b0ade35a65485a288648
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
2edc986847e209b4016e141a6dc8716d3207350f416969382d431539bf292e4a`
 3. Chunk signature:
 4. `0055627c9e194cb4542bae2aa5492e3c1575bbb81b612b7d234b86a503ef5497`
 5. Chunk data sent:
 6. `400;chunk-signature=0055627c9e194cb4542bae2aa5492e3c1575bbb81b612b7d234b86a503ef5497<1024 bytes>`
11. **Chunk 3: (0 byte data)**
 1. Chunk string to sign:
 2. `OWS1-HMAC-SHA256-PAYLOAD
20160401T000000Z
20160401/onepay/tsp/ows1_request
0055627c9e194cb4542bae2aa5492e3c1575bbb81b612b7d234b86a503ef5497
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855`
 3. Chunk signature:
 4. `b6c6ea8a5354eaf15b3cb7646744f4275b71ea724fed81ceb9323e279d449df9`
 5. Chunk data sent:
 6. `0;chunk-signature=b6c6ea8a5354eaf15b3cb7646744f4275b71ea724fed81ceb9323e279d449df9`

Authenticating Requests: Using Query Parameters (OWS Signature Version 1)

As described in the authentication overview (see Authentication Methods), you can provide authentication information using query string parameters. Using query parameters to authenticate requests is useful when you want to express a request entirely in a URL. This method is also referred as presigning a URL. A use case scenario for presigned URLs is that you can grant temporary access to your TSP resources. For example, you can embed a presigned URL on your website or alternatively use it in command line client (such as Curl) to download objects.

The following is an example presigned URL.

```
https://onepay.vn/examplebucket/test.txt
?X-OP-Algorithm=OWS1-HMAC-SHA256
&X-OP-Credential=<your-access-key-id>/20130721/onepay/tsp/ows1_request
&X-OP-Date=20160401T201207Z
&X-OP-Expires=86400
&X-OP-SignedHeaders=host
&X-OP-Signature=<signature-value>
```

In the example URL, note the following:

- The line feeds are added for readability.
- The X-OP-Credential value in the URL shows the "/" character only for readability. In practice, it should be encoded as %2F. For example:
- &X-OP-Credential=<your-access-key-id>%2F20130721%2Fonepay%2Ftsp%2Fows1_request

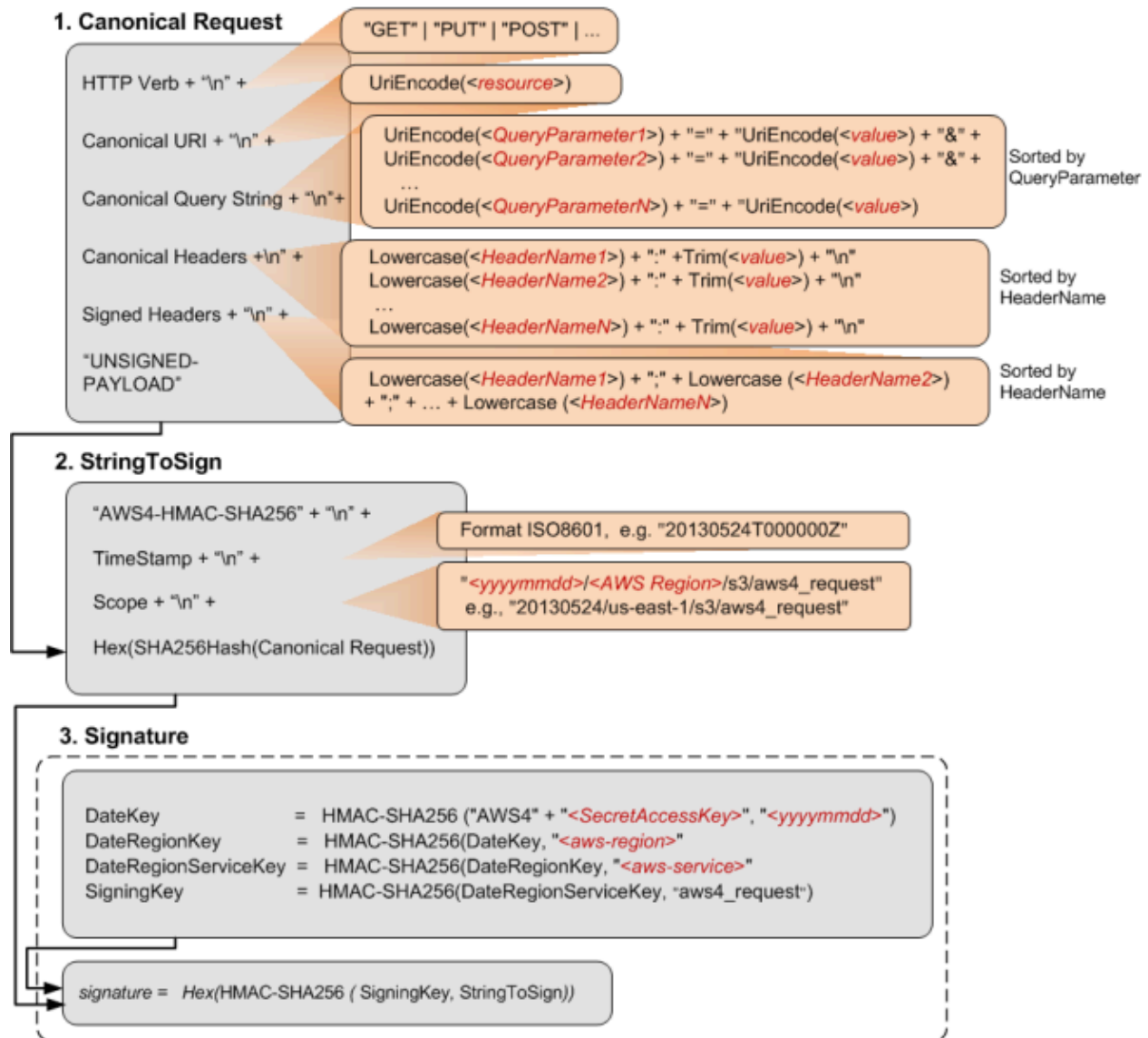
The following table describes the query parameters in the URL that provide authentication information.

Query String Parameter Name	Example Value
<i>X-OP-Algorithm</i>	Identifies the version of OWS Signature and the algorithm that you used to calculate the signature. For OWS Signature Version 1, you set this parameter value to OWS1-HMAC-SHA256. This string identifies OWS Signature Version 1 (OWS1) and the HMAC-SHA256 algorithm (HMAC-SHA256).
<i>X-OP-Credential</i>	In addition to your access key ID, this parameter also provides scope (OWS region and service) for which the signature is valid. This value must match the scope you use in signature calculations, discussed in the following section. The general form for this parameter value is as follows: <pre><your-access-key-id>/<date>/<OWS-region>/<OWS-service>/ows1_request</pre> For example: <pre>AKIAIOSFODNN7EXAMPLE/20130721/onepay/tsp/ows1_request</pre> For TSP, the <i>OWS-service</i> string is <i>tsp</i>. For a list of TSP OWS-region strings, see Regions and Endpoints in the <i>OWS General Reference</i>.

<i>X-OP-Date</i>	The date in ISO 8601 format, for example, 20160401T201207Z. This value must match the date value used to calculate the signature.
<i>X-OP-Expires</i>	Provides the time period, in seconds, for which the generated presigned URL is valid. For example, 86400 (24 hours). This value is an integer. The minimum value you can set is 1, and the maximum is 604800 (seven days). A presigned URL can be valid for a maximum of seven days because the signing key you use in signature calculation is valid for up to seven days.
<i>X-OP-SignedHeaders</i>	Lists the headers that you used to calculate the signature. The following headers are required in the signature calculations: • The HTTP <code>host</code> header. • Any <code>x-op-*</code> headers that you plan to add to the request. Note For added security, you should sign all the request headers that you plan to include in your request.
<i>X-OP-Signature</i>	Provides the signature to authenticate your request. This signature must match the signature TSP calculates; otherwise, TSP denies the request. For example, 733255ef022bec3f2a8701cd61d4b371f3f28c9f193a1f02279211d48d5193d7 Signature calculations are described in the following section.

Calculating a Signature

The following diagram illustrates the signature calculation process.



The following table describes the functions that are shown in the diagram. You need to implement code for these functions.

Function	Description
<code>Lowercase()</code>	Convert the string to lowercase.
<code>Hex()</code>	Lowercase base 16 encoding.
<code>SHA256Hash()</code>	Secure Hash Algorithm (SHA) cryptographic hash function.

HMAC-SHA256()	Computes HMAC by using the SHA256 algorithm with the signing key provided. This is the final signature.
Trim()	Remove any leading or trailing whitespace.
UriEncode()	URI encode every byte. UriEncode() must enforce the following rules: • URI encode every byte except the unreserved characters: 'A'-'Z', 'a'-'z', '0'-'9', '-', '.', '_', and '~'. • The space character is a reserved character and must be encoded as "%20" (and not as "+"). • Each URI encoded byte is formed by a '%' and the two-digit hexadecimal value of the byte. • Letters in the hexadecimal value must be uppercase, for example "%1A". • Encode the forward slash character, '/', everywhere except in the object key name. For example, if the object key name is <code>photos/Jan/sample.jpg</code>, the forward slash in the key name is not encoded. Caution The standard UriEncode functions provided by your development platform may not work because of differences in implementation and related ambiguity in the underlying RFCs. We recommend that you write your own custom UriEncode function to ensure that your encoding will work. The following is an example uri-encode() function in Java. <pre> public static String UriEncode(CharSequence input, boolean encodeSlash) { StringBuilder result = new StringBuilder(); for (int i = 0; i < input.length(); i++) { char ch = input.charAt(i); if ((ch >= 'A' && ch <= 'Z') (ch >= 'a' && ch <= 'z') (ch >= '0' && ch <= '9') ch == '_' ch == '-' ch == '~' ch == '.') { result.append(ch); } else if (ch == '/') { </pre>

```

        result.append(encodeSlash ? "%2F" : ch);
    } else {
        result.append(toHexUTF8(ch));
    }
}
return result.toString();
}

```

For more information about the signing process (details of creating a canonical request, string to sign, and signature calculations), see [Signature Calculations for the Authorization Header: Transferring Payload in a Single Chunk \(OWS Signature Version 1\)](#). The process is generally the same except that the creation of **CanonicalRequest** in a presigned URL differs as follows:

- You don't include a payload hash in the **Canonical Request**, because when you create a presigned URL, you don't know the payload content because the URL is used to upload an arbitrary payload. Instead, you use a constant string `UNSIGNED-PAYLOAD`.
- The **Canonical Query String** must include all the query parameters from the preceding table except for `X-OP-Signature`.
- **Canonical Headers** must include the HTTP `host` header. If you plan to include any of the `x-op-*` headers, these headers must also be added for signature calculation. You can optionally add all other headers that you plan to include in your request. For added security, you should sign as many headers as possible.

An Example

Suppose you have an object `test.txt` in your `examplebucket` bucket. You want to share this object with others for a period of 24 hours (86400 seconds) by creating a presigned URL.

```

https://onepay.vn/examplebucket/test.txt
?X-OP-Algorithm=OWS1-HMAC-SHA256
&X-OP-Credential=AKIAIOSFODNN7EXAMPLE%2F20160401%2Fonepay%2Ftsp%2Fows1_request
&X-OP-Date=20160401T000000Z&X-OP-Expires=86400&X-OP-SignedHeaders=host
&X-OP-Signature=<signature-value>

```

The following steps illustrate first the signature calculations and then construction of the presigned URL.

The example makes the following additional assumptions:

- Request timestamp is `Fri, 01 Apr 2016 00:00:00 GMT`.
- The bucket is in the US East (N. Virginia) region, and the credential `Scope` and the `Signing Key` calculations use `onepay` as the region specifier. For more information, see [Regions and Endpoints in the OWS General Reference](#).

You can use this example as a test case to verify the signature that your code calculates; however, you must use the same bucket name, object key, time stamp, and the following example credentials:

Parameter	Value
OWSAccessKeyId	AKIAIOSFODNN7EXAMPLE
OWSSecretAccessKey	wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY

1. StringToSign

1. CanonicalRequest

2. GET

/test.txt

X-OP-Algorithm=OWS1-HMAC-SHA256&X-OP-Credential=AKIAIOSFODNN7EXAMPLE%2F20160401%2Foneway%2Ftsp%2Fows1_request&X-OP-Date=20160401T000000Z&X-OP-Expires=86400&X-OP-SignedHeaders=host
host:secure.oneway.vn

host

UNSIGNED-PAYLOAD

3. StringToSign

4. OWS1-HMAC-SHA256

20160401T000000Z

20160401/oneway/tsp/ows1_request

3bfa292879f6447bbcd47001decf97f4a54dc650c8942174ae0a9121cf58ad04

2. SigningKey

3. signing key = HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 (HMAC-SHA256 ("OWS1" + "<YourSecretAccessKey>", "20160401"), "oneway"), "tsp"), "ows1_request")

4. Signature

5. aeed9bbccd4d02ee5c0109b86d86835f995330da4c265957d157751f604d404

6. Now you have all information to construct a presigned URL. The resulting URL for this example is shown as follows (you can use this to compare your presigned URL):

7. https://secure.oneway.vn/test.txt?X-OP-Algorithm=OWS1-HMAC-SHA256&X-OP-Credential=AKIAIOSFODNN7EXAMPLE%2F20160401%2Foneway%2Ftsp%2Fows1_request&X-OP-Date=20160401T000000Z&X-OP-Expires=86400&X-OP-SignedHeaders=host&X-OP-Signature=aeed9bbccd4d02ee5c0109b86d86835f995330da4c265957d157751f604d404